MANUAL DE INSTALACIÓN Y CONFIGURACIÓN DEL SOFTWARE

1. Introducción

- **Nombre del Software**: Roomix — Sistema Integral de Gestión Administrativa Hotelera.
- **Versión**: 1.1 (backend Spring Boot 4.0 + frontend Angular 21; aplicación Android con Capacitor 8).
- **Descripción**: aplicación web que centraliza la operación de un hotel: usuarios y roles, auditoría, panel del día, habitaciones, reservas, huéspedes, limpieza y mantenimiento, servicios al huésped, eventos, personal, inventario, finanzas, facturación y reportes. Se compone de un API REST (backend), una interfaz web (frontend) y una base de datos PostgreSQL.
- **Objetivo del Manual**: Este manual proporciona los pasos para instalar y configurar el software Roomix, tanto en un equipo local para desarrollo o demostración como en la nube para producción.
- **Conocimientos Previos**: manejo básico de la terminal (PowerShell o Bash), Git, conceptos de variables de entorno, PostgreSQL y, para el despliegue, nociones de Docker y servicios en la nube (Neon y Render).
- **Alcance**: cubre la instalación básica (local) y configuraciones avanzadas: despliegue en la nube, generación de la aplicación Android (APK), usuarios y roles, correo de recuperación de contraseña, datos del hotel, seguridad, actualizaciones, respaldo y desinstalación.

2. Requisitos del Sistema

- **Hardware**:
    - CPU: procesador de 64 bits de 2 núcleos (4 núcleos recomendado para compilar el APK).
    - RAM: 8 GB recomendado en el equipo de desarrollo (mínimo 4 GB); en producción el backend funciona con 512 MB.
    - Espacio en Disco: 5 GB para el código y las dependencias (Maven y npm); 10 GB adicionales si se instala Android Studio para generar el APK.
- **Software**:

- Sistema Operativo: Windows 10/11 (64 bits), Ubuntu 22.04 o superior, o macOS 13 o superior.
- Dependencias adicionales: JDK 21, PostgreSQL 14 o superior (probado con 18), Node.js 22 con npm 11, Git. Opcionales: Docker (para la imagen del backend), Android Studio / Android SDK (para el APK) y un navegador actualizado.

- **Entorno**:
  - Lenguaje de programación: Java 21 (Spring Boot 4.0, Spring Security, Spring Data JPA) en el backend y TypeScript (Angular 21, Angular Material) en el frontend.
  - Servidor web o de aplicaciones: Apache Tomcat embebido en Spring Boot (puerto 8080) para el API; el frontend es un sitio estático servido por Render Static Site, Nginx o cualquier servidor web (en desarrollo, ng serve en el puerto 4200).

3. Descarga del Software

- **Repositorio**: https://github.com/DarwinSt/Roomix-Back (backend) y https://github.com/DarwinSt/Roomix-FRONT (frontend). Imagen Docker del backend: sof220241004/roomix en DockerHub.
- **Versión recomendada**: la última versión de la rama main (backend) y master (frontend), que corresponde a Roomix 1.1 (pago obligatorio al reservar, abonos, promociones, políticas de reserva y gastos por pagar).
- **Instrucciones para la descarga**:
  - Paso 1: clonar el backend: git clone https://github.com/DarwinSt/Roomix-Back.git
  - Paso 2: clonar el frontend: git clone https://github.com/DarwinSt/Roomix-FRONT.git
  - Paso 3 (opcional, solo para ejecutar el backend con Docker): docker pull sof220241004/roomix

4. Instalación del Software

4.1 Instalación en Windows 10/11

- **Paso 1**: instalar JDK 21, PostgreSQL, Node.js 22 y Git (con los instaladores oficiales o con winget). Durante la instalación de PostgreSQL defina la contraseña del usuario postgres.
- **Paso 2**: abrir una terminal en cada proyecto e instalar sus dependencias: el backend las descarga el wrapper de Maven al compilar; en el frontend ejecute npm install.
- **Ejemplo de comando**: `winget install EclipseAdoptium.Temurin.21.JDK PostgreSQL.PostgreSQL.17 OpenJS.NodeJS.LTS Git.Git`
- **Compilar y verificar**: `cd Roomix-Back → .\mvnw.cmd -DskipTests package | cd Roomix-FRONT → npm install`

4.2 Instalación en Linux (Ubuntu 22.04+)

- **Paso 1**: instalar Java 21, PostgreSQL, Node.js 22 y Git desde los repositorios del sistema (Node.js desde NodeSource o nvm).
- **Paso 2**: habilitar el servicio de PostgreSQL, clonar los repositorios y ejecutar ./mvnw package en el backend y npm install en el frontend.
- **Ejemplo de comando**: `sudo apt install openjdk-21-jdk postgresql git && sudo systemctl enable --now postgresql`
- **Con Docker (cualquier sistema)**: `docker run -p 8080:8080 --env-file .env sof220241004/roomix`

4.3 Despliegue en la nube (Neon + Render)

- **Paso 1**: crear la base de datos en Neon (neon.tech) y copiar la cadena de conexión (Connection string).
- **Paso 2**: en Render crear un Web Service con la imagen Docker sof220241004/roomix (o desde el repositorio Roomix-Back con su Dockerfile), definir las variables de entorno de la sección 5 y el Health Check Path /actuator/health.
- **Paso 3**: en Render crear un Static Site desde Roomix-FRONT con Build Command npm ci && npm run build, Publish Directory dist/roomix/browser, variable API_URL=https://<backend>.onrender.com/api y la regla Rewrite /* → /index.html. En el Static Site, en «Redirects/Rewrites», agregar la regla Source /*, Destination /index.html, Action Rewrite: sin ella, al refrescar (F5) una pantalla como /habitaciones/2/reservar aparece «Not Found». Como respaldo, npm run build también genera 404.html (scripts/spa-fallback.mjs).

- **Paso 4**: agregar la URL del frontend a CORS_ALLOWED_ORIGINS y a FRONTEND_URL en el backend y redesplegar.
- **Ejemplo de comando**: `docker build -t sof220241004/roomix:latest . && docker push sof220241004/roomix:latest`

4.4 Generación de la aplicación Android (APK)

- **Paso 1**: instalar Android Studio (incluye el Android SDK) y definir JAVA_HOME (JDK 21) y ANDROID_HOME.
- **Paso 2**: en Roomix-FRONT compilar apuntando al backend de producción y generar el APK; el archivo queda en android/app/build/outputs/apk/debug/app-debug.apk. Para publicarlo, firmar la versión release desde Android Studio.
- **Ejemplo de comando**: `API_URL=https://roomix-0-0-1.onrender.com/api npm run apk:debug`
- **Nota**: el proyecto usa Capacitor directamente y no es un proyecto Ionic, por lo que los comandos del CLI de Ionic (ionic serve, ionic build, ionic cap ...) responden «can only be run in an Ionic project directory». Equivalentes: npm start (ionic serve), npm run build (ionic build), npx cap sync android (ionic cap sync), npm run android:open (ionic cap open android) y npx cap run android (ionic cap run android). En el terminal de Windows (cmd): set API_URL=https://roomix-0-0-1.onrender.com/api&& npm run build:android y luego cd android && gradlew assembleDebug.

5. Configuración Inicial

- **Configuración de la base de datos**:
    - Crear base de datos con el siguiente comando: CREATE DATABASE roomix ENCODING 'UTF8'; (en la nube, Neon la crea al registrar el proyecto). Las tablas las crea el backend automáticamente al iniciar.
    - Configurar las credenciales de conexión en el archivo de configuración: variables de entorno del backend (archivo .env o panel Environment de Render), con DATABASE_URL o el trío DB_URL / DB_USERNAME / DB_PASSWORD. En local también puede usarse src/main/resources/application-local.properties (no se sube al repositorio).
    - Ejemplo de archivo de configuración (plantilla completa en Roomix-Back/.env.example):

Roomix-Back/.env (variables de entorno del backend)

```
# Base de datos (Neon / Railway / Render o local)
DATABASE_URL=postgresql://usuario:clave@host:5432/roomix?sslmode=require
# Seguridad
JWT_SECRET=texto-aleatorio-de-al-menos-32-caracteres
ADMIN_USERNAME=admin
ADMIN_PASSWORD=UnaClaveSegura2026
APP_DEMO_USERS=true
# Frontend permitido (CORS) y enlaces de correo
CORS_ALLOWED_ORIGINS=https://roomix-front-lnoq.onrender.com,https://localhost
FRONTEND_URL=https://roomix-front-lnoq.onrender.com
# Datos de demostración (solo el primer despliegue)
APP_SEED_ENABLED=true
# Opcionales
APP_TIMEZONE=America/Bogota
HOTEL_NOMBRE=Hotel Roomix
HOTEL_IVA=19
```

- **Configuración del servidor**:
    - Editar las variables PORT (puerto del API, por defecto 8080; Render lo asigna automáticamente) y CORS_ALLOWED_ORIGINS (direcciones del frontend autorizadas). En el frontend, la variable API_URL define la dirección del API durante npm run build.
    - Instrucciones específicas para cada tipo de servidor: Render Static Site requiere la regla Rewrite /* → /index.html; en Nginx use location / { try_files $uri $uri/ /index.html; } y sirva la carpeta dist/roomix/browser; en Apache habilite mod_rewrite con una regla equivalente hacia index.html. Si falta la regla, la aplicación abre bien desde el inicio, pero al refrescar una ruta interna el servidor responde «Not Found».
    - Datos de demostración: con APP_SEED_ENABLED=true el backend carga un hotel de ejemplo (30 habitaciones, reservas, facturas, eventos e inventario) solo si la base no tiene habitaciones.
    - Políticas de reserva y gastos recurrentes: al primer arranque la política queda con abono mínimo de 0 % (basta cualquier pago para reservar) y penalidad de 0 %; el administrador la define en Promociones → Políticas de reserva. Los gastos recurrentes generan su egreso por pagar del mes al iniciar el backend y cada día a las 00:15 (hora del servidor); no requieren configuración adicional.

6. Ejecución del Software

- **Paso 1**: iniciar el backend con .\mvnw.cmd spring-boot:run (o mvnw spring-boot:run en Linux) desde Roomix-Back y, en otra terminal, el frontend con npm start desde Roomix-FRONT.
- **Paso 2**: acceder al software desde el navegador en http://localhost:4200 (producción: https://roomix-front-lnoq.onrender.com) e ingresar con el usuario admin y la contraseña definida en ADMIN_PASSWORD.
- **Comprobación**: verificar que http://localhost:8080/actuator/health responda {"status":"UP"}, que la documentación del API abra en http://localhost:8080/swagger-ui.html y que la pantalla de inicio de sesión cargue en el navegador.

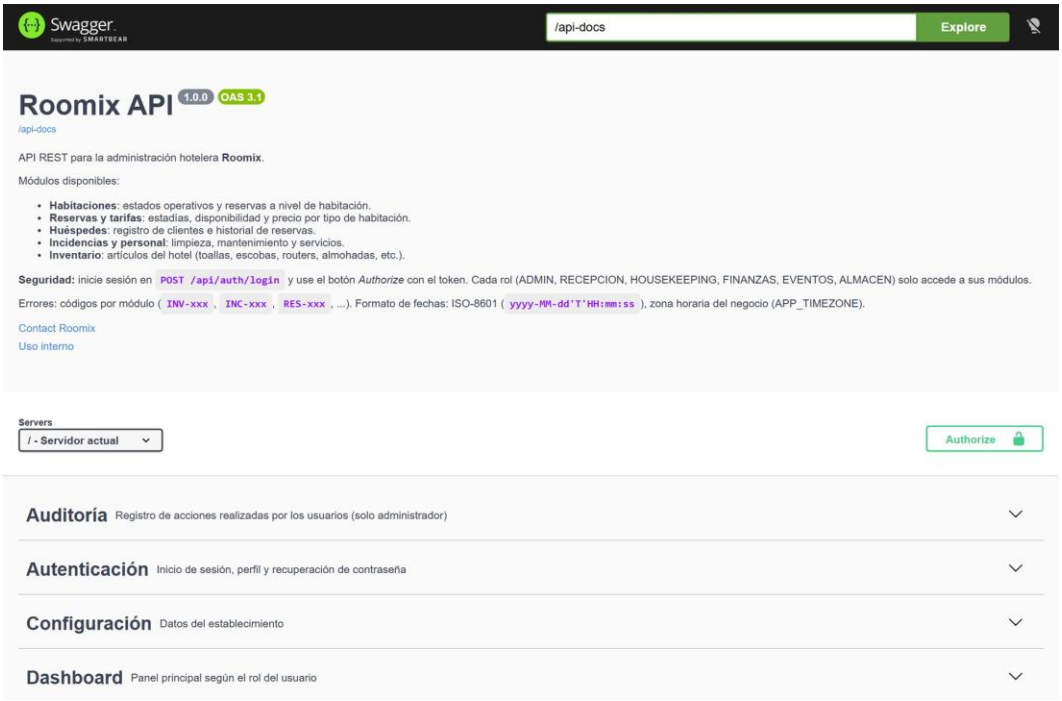


Figura 1. Documentación del API (Swagger UI) en ejecución

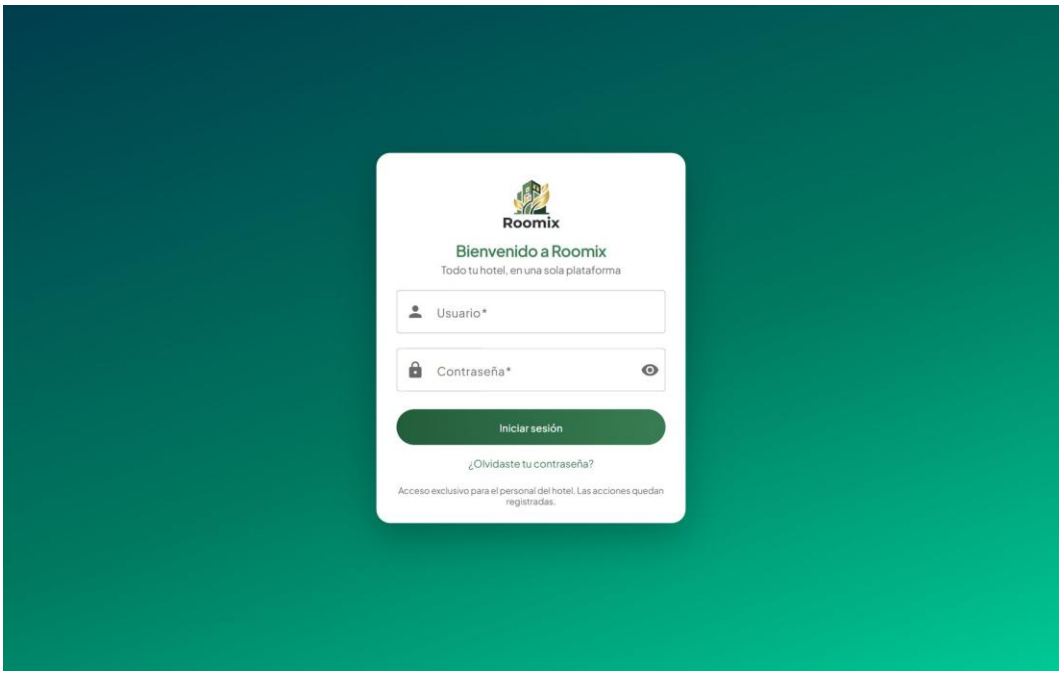


Figura 2. Pantalla de inicio de sesión de Roomix

7. Configuración Avanzada (Opcional)

- **Roles de usuario y permisos**: al primer arranque se crea el administrador (ADMIN_USERNAME / ADMIN_PASSWORD) y, si APP_DEMO_USERS=true, un usuario por rol (recepcion, housekeeping, finanzas, eventos, almacen) con DEMO_USERS_PASSWORD. Los seis roles (Administrador, Recepción, Housekeeping, Finanzas, Eventos y Almacén) tienen permisos fijos validados por el servidor; el administrador crea usuarios, asigna roles, desactiva cuentas y restablece contraseñas desde el módulo Usuarios.

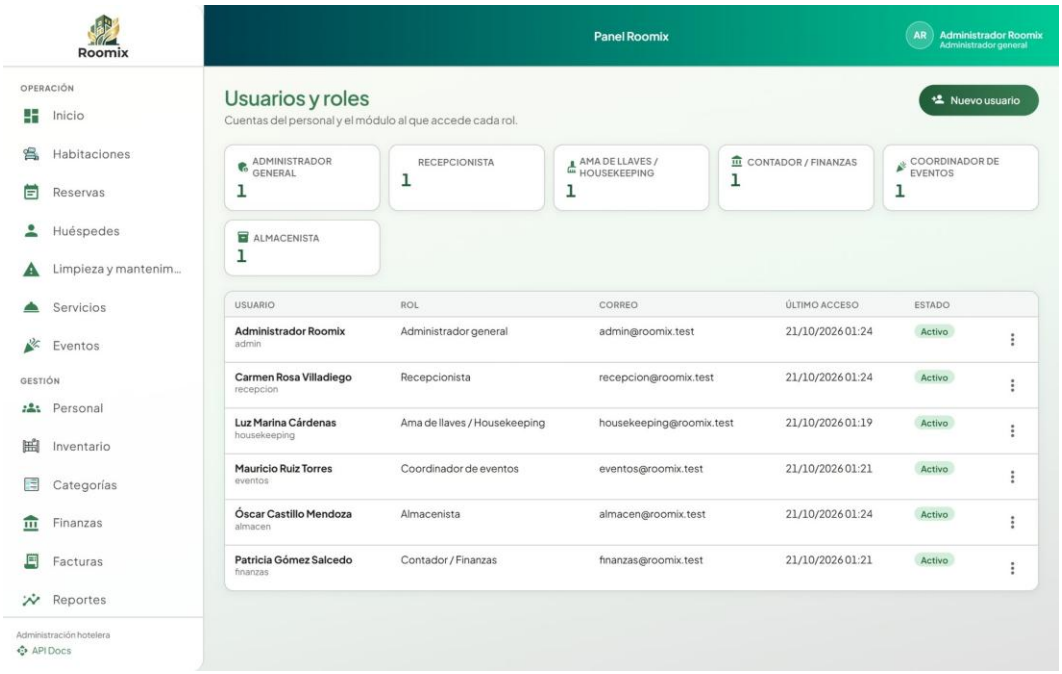


Figura 3. Módulo de usuarios y roles

- **Integraciones externas**: correo SMTP para la recuperación de contraseña (SPRING_MAIL_HOST, SPRING_MAIL_PORT, SPRING_MAIL_USERNAME, SPRING_MAIL_PASSWORD y MAIL_FROM; con Gmail use una contraseña de aplicación). Base de datos Neon, alojamiento en Render, imágenes en DockerHub y despliegue automático con el Deploy Hook de Render.
- **Ajustes adicionales**: seguridad: JWT_SECRET (obligatorio en producción) y JWT_EXPIRATION_HOURS (por defecto 8); rendimiento: DB_POOL_SIZE (por defecto 5); documentación: SWAGGER_ENABLED=false para ocultar Swagger; zona horaria: APP_TIMEZONE; facturación: HOTEL_NOMBRE, HOTEL_NIT, HOTEL_DIRECCION, HOTEL_CIUDAD, HOTEL_TELEFONO, HOTEL_EMAIL, HOTEL_IVA y HOTEL_EXENTO_IVA_EXTRANJEROS.

8. Actualizaciones y Mantenimiento

- Paso 1: actualizar el código con git pull en cada repositorio; en la nube, construir y subir la nueva imagen (docker build / docker push) y redesplegar el servicio en Render. El frontend se redespliega al subir cambios a GitHub.
- Paso 2: realizar copias de seguridad de la base de datos antes de actualizar: pg_dump "$DATABASE_URL" > respaldo.sql (Neon además permite restaurar a un punto en el tiempo).

- Paso 3: no se requieren scripts manuales de migración: al iniciar, Hibernate ajusta el esquema (JPA_DDL_AUTO=update) y las clases *Migration del backend completan los cambios de datos. Verificar /actuator/health tras el despliegue. La versión 1.1 agrega las tablas abonos_reserva, promociones, politica_reserva y gastos_recurrentes y columnas nuevas en reservas, pagos y movimientos_financieros; Hibernate las crea en el primer arranque. Despliegue el backend y el frontend de la misma versión: un frontend 1.1 con un backend 1.0 no puede calcular el pago de la reserva.

9. Desinstalación

- Paso 1: detener los servicios: Ctrl + C en las terminales del backend y del frontend, docker stop <contenedor> si usa Docker, o Suspend / Delete del servicio en Render.
- Paso 2: eliminar las carpetas Roomix-Back y Roomix-FRONT (y la aplicación Roomix del teléfono, si se instaló el APK).
- Paso 3: eliminar la base de datos creada con el comando: DROP DATABASE roomix; (en Neon: Settings → Delete project).

10. Solución de Problemas (Troubleshooting)

- **Problema 1**: El frontend muestra «Error 0» o no conecta con el servidor
    - **Solución**: agregar la URL exacta del frontend (sin barra final) a CORS_ALLOWED_ORIGINS del backend y redesplegar; verificar API_URL del frontend.
- **Problema 2**: El backend no inicia: «password authentication failed» o «Connection refused»
    - **Solución**: revisar DATABASE_URL o DB_URL / DB_USERNAME / DB_PASSWORD (que el usuario y la contraseña no estén intercambiados) y que PostgreSQL esté en ejecución.
- **Problema 3**: Las sesiones se cierran cada vez que se reinicia el backend
    - **Solución**: definir JWT_SECRET con un valor fijo de al menos 32 caracteres.
- **Problema 4**: «Port 8080 was already in use»
    - **Solución**: detener el proceso que ocupa el puerto o iniciar con otra variable PORT (y ajustar API_URL del frontend).

- **Problema 5**: La primera carga en Render tarda cerca de un minuto
    - **Solución**: es normal en el plan gratuito, que suspende el servicio sin tráfico; un plan pago o un monitor de disponibilidad lo mantiene activo.
- **Problema 6**: El APK no inicia sesión
    - **Solución**: compilarlo con API_URL apuntando al backend público (https) e incluir https://localhost en CORS_ALLOWED_ORIGINS.
- **Problema 7**: No llega el correo de recuperación de contraseña
    - **Solución**: configurar las variables SPRING_MAIL_* y MAIL_FROM; sin SMTP el enlace se escribe en el log del backend.
- **Problema 8**: Al refrescar (F5) una pantalla del sitio desplegado aparece «Not Found»
    - **Solución**: agregar en Render la regla Rewrite /* → /index.html (sección 4.3) y volver a publicar el frontend con npm run build, que genera 404.html.
- **Problema 9**: «ionic serve can only be run in an Ionic project directory»
    - **Solución**: el proyecto no usa el CLI de Ionic; ejecutar npm start para el sitio local y los comandos de Capacitor de la sección 4.4 para el APK.
- **Problema 10**: Al reservar aparece «No se pudo calcular el total y el pago» o «Para reservar debe registrar el pago»
    - **Solución**: la primera indica que el backend no responde /api/reservas/cotizacion (versión anterior a la 1.1 o servidor caído): desplegar el backend 1.1. La segunda es la regla de negocio: registrar el pago completo o un abono igual o mayor al mínimo.

11. Soporte y Contacto

- **Correo de soporte**: david_perdomora@fet.edu.co, darwin_moraga@fet.edu.co, camilo_acostahe@fet.edu.co
- **Documentación oficial**: README de cada repositorio, carpeta docs/ del proyecto (manual técnico, manual de usuario, casos de prueba) y documentación del API en /swagger-ui.html.

- **Foros de soporte**: sección Issues de los repositorios en GitHub (https://github.com/DarwinSt/Roomix-Back/issues y https://github.com/DarwinSt/Roomix-FRONT/issues).

12. Glosario (Opcional)

- **API REST**: interfaz que expone las operaciones del backend mediante peticiones HTTP y respuestas JSON.
- **CORS**: mecanismo del navegador que solo permite llamar al API desde los dominios autorizados.
- **Docker**: herramienta que empaqueta la aplicación con todo lo necesario para ejecutarse igual en cualquier servidor.
- **JWT**: token firmado que identifica al usuario en cada petición tras iniciar sesión.
- **Neon**: servicio de PostgreSQL administrado en la nube.
- **Render**: plataforma en la nube donde se publican el backend (Docker) y el frontend (sitio estático).
- **Variable de entorno**: valor de configuración que se define fuera del código (contraseñas, URL, claves).
- **APK**: paquete instalable de una aplicación Android.

13. Licencia y Créditos

- **Licencia del software**: uso académico — proyecto de grado de la Fundación Escuela Tecnológica de Neiva «Jesús Oviedo Pérez» (FET). [Ajustar si el equipo define una licencia abierta, p. ej. MIT.]
- **Créditos**: David Francisco Perdomo Ramon, Darwin Steween Mora Gamboa y Camilo Andres Acosta Hernandez — Programa Tecnología en Sistemas de Información, FET, Neiva (Huila), 2026.